BIBLIOTECAS

Geralmente, bibliotecas são pedaços de código escritos por você ou por outras pessoas que podem ser usados em seu programa.

Python permite que você compartilhe funções ou recursos com outras pessoas como “módulos”.

Se você copiar e colar o código de um projeto antigo, é provável que você possa criar um módulo ou biblioteca que possa trazer para o seu novo projeto.

RANDOM - ALEATÓRIO

Random é uma biblioteca que vem com Python que você pode importar para seu próprio projeto.

É mais fácil para um programador apoiar-se nos ombros dos programadores anteriores.

Então, como você carrega um módulo em seu próprio programa? Você pode usar a palavra importar em seu programa.

Dentro do módulo aleatório, existe uma função integrada chamada random.choice(seq). random é o módulo que você está importando. Dentro desse módulo, existe a função de escolha. Essa função leva consigo uma sequência ou sequência que é uma lista.

Na janela do seu terminal, digite o código generate.py. Em seu editor de texto, codifique da seguinte forma:

```
import random


coin = random.choice(["heads", "tails"])

print(coin)
```

Observe que a lista dentro da escolha tem colchetes, aspas e vírgula. Como você passou em dois itens, Python faz as contas e dá 50% de chance de cara e coroa. Executando seu código, você notará que ele realmente funciona bem!

Podemos melhorar nosso código. from nos permite ser muito específicos sobre o que gostaríamos de importar. Antes, nossa linha de código de importação está trazendo todo o conteúdo das funções aleatórias. Porém, e se quisermos carregar apenas uma pequena parte de um módulo? Modifique seu código da seguinte maneira:

```
from random import choice
```

```
coin = choice(["heads", "tails"])

print(coin)
```

Observe que agora podemos importar apenas a função de escolha aleatória. Desse ponto em diante, não precisamos mais codificar random.choice. Agora só podemos escolher o código. escolha é carregada explicitamente em nosso programa. Isso economiza recursos do sistema e potencialmente pode fazer nosso código rodar mais rápido!

Continuando, considere a função random.randint(a, b). Esta função irá gerar um número aleatório entre a e b. Modifique seu código da seguinte maneira:

```
import random


number = random.randint(1, 10)

print(number)
```

Observe que nosso código irá gerar aleatoriamente um número entre 1 e 10.

Podemos introduzir em nosso cartão random.shuffle(x) onde ele embaralhará uma lista em uma ordem aleatória.

```
import random


cards = ["jack", "queen", "king"]

random.shuffle(cards)

for card in cards:

  print(card)
```

Observe que random.shuffle embaralhará as cartas no lugar. Ao contrário de outras funções, não retornará um valor. Em vez disso, ele pegará a lista de cartas e as embaralhará dentro dessa lista. Execute seu código algumas vezes para ver o código funcionando.

Agora temos essas três maneiras acima de gerar informações aleatórias.

ESTATÍSTICAS

Python vem com uma biblioteca de estatísticas integrada. Como podemos usar este módulo?

média é uma função desta biblioteca que é bastante útil. Na janela do terminal, digite o código Average.py. Na janela do editor de texto, modifique seu código da seguinte maneira:

```
import statistics

print(statistics.mean([100, 90]))
```

Observe que importamos uma biblioteca diferente chamada estatística. A função média recebe uma lista de valores. Isso imprimirá a média desses valores. Na janela do terminal, digite python Average.py.

Considere as possibilidades de usar o módulo de estatísticas em seus próprios programas.

ARGUMENTOS DE LINHA DE COMANDO

Até o momento, disponibilizamos todos os valores dentro do programa que criamos. E se quiséssemos receber informações da linha de comando? Por exemplo, em vez de digitar python Average.py no terminal, e se quiséssemos digitar python Average.py 100 90 e obter a média entre 100 e 90?

Sys é um módulo que nos permite receber argumentos na linha de comando.

Argv é uma função do módulo sys que nos permite aprender o que o usuário digitou na linha de comando. Observe como você verá sys.argv utilizado no código abaixo. Na janela do terminal, digite o code name.py. No editor de texto, codifique da seguinte forma:

```
import sys

print("hello, my name is", sys.argv[1])
```

Observe que o programa analisará o que o usuário digitou na linha de comando. Atualmente, se você digitar python name.py David na janela do terminal, verá olá, meu nome é David. Observe que sys.argv[1] é onde David está sendo armazenado. Por que isso? Bem, nas lições anteriores, você deve se lembrar que as listas começam no 0º elemento. O que você acha que está armazenado atualmente em sys.argv[0]? Se você adivinhou name.py, você acertou!

Há um pequeno problema com o nosso programa tal como está. E se o usuário não digitar o nome na linha de comando? Tente você mesmo. Digite python name.py na janela do terminal. Um índice de lista de erros fora do intervalo será apresentado pelo compilador. A razão para isso é que não há nada em sys.argv[1] porque nada foi digitado! Veja como podemos proteger nosso programa contra esse tipo de erro:

```
import sys

try:
    print("hello, my name is", sys.argv[1])
except IndexError:
    print("Too few arguments")
```

Observe que o usuário agora receberá uma dica útil sobre como fazer o programa funcionar caso se esqueça de digitar um nome. No entanto, poderíamos ser mais defensivos para garantir que o usuário insira os valores corretos?

Nosso programa pode ser melhorado da seguinte forma:

```
import sys

if len(sys.argv) < 2:
    print("Too few arguments")
elif len(sys.argv) > 2:
    print("Too many arguments")
else:
    print("hello, my name is", sys.argv[1])
```

Observe como, se você testar seu código, verá como essas exceções são tratadas, fornecendo ao usuário conselhos mais refinados. Mesmo que o usuário digite muitos ou poucos argumentos, ele receberá instruções claras sobre como corrigir o problema.

No momento, nosso código está logicamente correto. No entanto, há algo muito bom em manter nossa verificação de erros separada do restante do nosso código. Como poderíamos separar nosso tratamento de erros? Modifique seu código da seguinte maneira:

```
import sys

if len(sys.argv) < 2:
    sys.exit("Too few arguments")
elif len(sys.argv) > 2:
    sys.exit("Too many arguments")

print("hello, my name is", sys.argv[1])
```

Observe como estamos usando uma função integrada do sys chamada exit que nos permite sair do programa se um erro for introduzido pelo usuário. Podemos ter certeza agora de que o programa nunca executará a linha final do código e disparará um erro. Portanto, sys.argv fornece uma maneira pela qual os usuários podem introduzir informações a partir da linha de comando. sys.exit fornece um meio pelo qual o programa pode sair se ocorrer um erro.

SLICE

Slice é um comando que nos permite pegar uma lista e informar ao compilador onde queremos que o compilador considere o início e o fim da lista. Por exemplo, modifique seu código da seguinte maneira:

```
import sys

if len(sys.argv) < 2:
    sys.exit("Too few arguments")

for arg in sys.argv:
    print("hello, my name is", arg)
```

Observe que se você digitar python name.py David Carter Rongxin na janela do terminal, o compilador produzirá não apenas a saída pretendida dos nomes, mas também olá, meu nome é name.py. Como então poderíamos garantir que o compilador ignore o primeiro elemento da lista onde name.py está sendo armazenado atualmente?

Slice pode ser empregado em nosso código para iniciar a lista em algum lugar diferente! Modifique seu código da seguinte maneira:

```
import sys

if len(sys.argv) < 2:
    sys.exit("Too few arguments")

for arg in sys.argv[1:]:
    print("hello, my name is", arg)
```

Observe que em vez de iniciar a lista em 0, usamos colchetes para dizer ao compilador para começar em 1 e ir até o final usando o argumento 1:. Executando este código, você notará que podemos melhorar nosso código usando uma sintaxe relativamente simples.

PACOTES – PACKAGES

Uma das razões pelas quais o Python é tão popular é que existem inúmeras bibliotecas poderosas de terceiros que adicionam funcionalidades. Chamamos essas bibliotecas de terceiros, implementadas como uma pasta, de “pacotes”.

PyPI é um repositório ou diretório de todos os pacotes de terceiros disponíveis atualmente.

cowsay é um pacote bem conhecido que permite que uma vaca fale com o usuário.

Python possui um gerenciador de pacotes chamado pip que permite instalar pacotes rapidamente em seu sistema.

Na janela do terminal, você pode instalar o pacote cowsay digitando pip install cowsay. Depois de um pouco de saída, agora você pode usar este pacote em seu código.

Na janela do terminal, digite o código say.py. No editor de texto, codifique da seguinte forma:

```
import cowsay
import sys
```

```
if len(sys.argv) == 2:
    cowsay.cow("hello, " + sys.argv[1])
```

Observe que o programa primeiro verifica se o usuário inseriu pelo menos dois argumentos na linha de comando. Então, a vaca deverá falar com o usuário. Digite python say.py David e você verá uma vaca dizendo “olá” para David.

Modifique ainda mais seu código:

```
import cowsay
import sys

if len(sys.argv) == 2:
    cowsay.trex("hello, " + sys.argv[1])
```

Observe que um t-rex agora está dizendo “olá”.

APIS

APIs ou “interfaces de programas de aplicativos” permitem que você se conecte ao código de outras pessoas.

requests é um pacote que permite que seu programa se comporte como um navegador da web faria.

No seu terminal, digite pip install requests. Em seguida, digite o código itunes.py.

Acontece que o Apple iTunes tem sua própria API que você pode acessar em seus programas. No seu navegador de internet, você pode visitar https://itunes.apple.com/search?entity=song&limit=1&term=weezer e um arquivo de texto será baixado. David construiu este URL lendo a documentação da API da Apple. Observe como esta consulta procura uma música, com limite de um resultado, que se relacione com o termo chamado weezer. Olhando para este arquivo de texto baixado, você pode descobrir que o formato é semelhante ao que programamos anteriormente em Python.

O formato no arquivo de texto baixado é chamado JSON, um formato baseado em texto usado para trocar dados baseados em texto entre aplicativos. Literalmente, a Apple está fornecendo um arquivo JSON que poderíamos interpretar em nosso próprio programa Python.

Na janela do terminal, digite o código itunes.py. Codifique da seguinte forma:

```
import requests
import sys

if len(sys.argv) != 2:
    sys.exit()

response = requests.get("https://itunes.apple.com/search?entity=song&limit=1&term=" + sys.argv[1])
print(response.json())
```

Observe como o valor retornado de requests.get será armazenado na resposta. David, depois de ler a documentação da Apple sobre esta API, sabe que o que é retornado é um arquivo JSON. Executando python itunes.py weezer, você verá o arquivo JSON retornado pela Apple. No entanto, a resposta JSON é convertida pelo Python em um dicionário. Olhando para o resultado, pode ser bastante estonteante!

Acontece que o Python possui uma biblioteca JSON integrada que pode nos ajudar a interpretar os dados recebidos. Modifique seu código da seguinte maneira:

```
import json
import requests
import sys

if len(sys.argv) != 2:
    sys.exit()

response = requests.get("https://itunes.apple.com/search?entity=song&limit=1&term=" + sys.argv[1])
print(json.dumps(response.json(), indent=2))
```

Observe que json.dumps é implementado de forma que utiliza recuo para tornar a saída mais legível. Executando python itunes.py weezer, você verá o mesmo arquivo JSON. Porém, desta vez, é muito mais legível. Observe agora que você verá um dicionário chamado resultados dentro da saída. Dentro desse dicionário chamado resultados existem inúmeras chaves presentes. Observe o valor trackName na saída. Que nome de faixa você vê em seus resultados?

Como poderíamos simplesmente exibir apenas o nome daquela faixa? Modifique seu código da seguinte maneira:

```
import json
import requests
import sys

if len(sys.argv) != 2:
    sys.exit()

response = requests.get("https://itunes.apple.com/search?entity=song&limit=50&term=" + sys.argv[1])

o = response.json()
for result in o["results"]:
    print(result["trackName"])
```

Observe como pegamos o resultado de response.json() e o armazenamos em o (como na letra minúscula). Em seguida, estamos iterando os resultados em o e imprimindo cada trackName. Observe também como aumentamos o número limite de resultados para 50. Execute seu programa. Veja os resultados.

CRIANDO SUAS PRÓPRIAS BIBLIOTECAS

Você, como programador Python, tem a capacidade de criar sua própria biblioteca!

Imagine situações em que você queira reutilizar pedaços de código repetidamente ou até mesmo compartilhá-los com outras pessoas!

Escrevemos muitos códigos para dizer “olá” até agora nesta disciplina. Vamos criar um pacote que nos permita dizer “olá” e “tchau”. Na janela do terminal, digite o código provérbios.py. No editor de texto, codifique da seguinte forma:

```
def hello(name):
    print(f"hello, {name}")


def goodbye(name):
    print(f"goodbye, {name}")
```

Observe que esse código por si só não faz nada pelo usuário. Porém, se um programador importasse este pacote para seu próprio programa, as habilidades criadas pelas funções acima poderiam ser implementadas em seu código.

Vamos ver como poderíamos implementar código utilizando este pacote que criamos. Na janela do terminal, digite o código say.py. Neste novo arquivo em seu editor de texto, digite o seguinte:

```
import sys


from saying import goodbye


if len(sys.argv) == 2:
    goodbye(sys.argv[1])
```

Observe que este código importa as habilidades de adeus do pacote de provérbios. Se o usuário inseriu pelo menos dois argumentos na linha de comando, ele dirá “adeus” junto com a string inserida na linha de comando.

RESUMINDO

As bibliotecas ampliam as habilidades do Python. Algumas bibliotecas são incluídas por padrão no Python e simplesmente precisam ser importadas. Outros são pacotes de terceiros que precisam ser instalados usando pip. Você pode fazer seus próprios pacotes para uso próprio ou de outras pessoas! Nesta aula você aprendeu sobre…

- Bibliotecas - Libraries
- Aleatório - Random
- Estatística - Statistics
- Argumentos de Linha - Command-Line Arguments
- Fatia - Slice
- Pacotes - Packages
- APIs
- Criando suas Próprias Bibliotecas